



Research paper

DeFinder: Error-sensitive testing of deep neural networks via vulnerability interpretation

Aoshuang Ye^{a,b}, Shilin Zhang^a, Benxiao Tang^a, Jianpeng Ke^c, Yiru Zhao^c,
Tao Peng^{a,*}^a School of Computer Science and Artificial Intelligence, Wuhan Textile University, Wuhan, 430200, China^b Advanced Cryptography and System Security Key Laboratory of Sichuan Province, Sichuan, China^c School of Cyber Science and Engineering, Wuhan University, Wuhan, 430072, China

ARTICLE INFO

Keywords:

DNN testing

Test generation

Coverage

Model vulnerability

Fault detection

ABSTRACT

DNN testing evaluates the vulnerability of neural networks through *adversarial test cases*. The developers implement minor perturbations to the seed inputs to generate test cases, which are guided by meticulously designed testing criteria. Nevertheless, current coverage-guided testing methods rely on covering model states rather than analyzing the influence of seed inputs on inducing erroneous behaviors. In this paper, we propose a novel DNN testing method called DeFinder, which generates error-sensitive tests by implementing an explainable framework for neural networks to establish correlations between model vulnerability and seed inputs. By systematically analyzing vulnerable regions within seed inputs, DeFinder significantly improves the test suite's ability to maximize test coverage and expose errors. To validate the effectiveness of DeFinder, we conduct comprehensive experiments with nine deep neural network models from two popular computer vision datasets. We compare the proposed method with several state-of-the-art DNN testing tools. The experimental results demonstrate that DeFinder improves the error-triggering ratio by up to 58% and increases test coverage by up to 4.3%. For reproducibility, the artifact for this work is available at public repository: <https://github.com/Konatazz/DeFinder>.

1. Introduction

The inherent vulnerabilities of deep neural networks (DNNs) to adversarial inputs demonstrate urgent security risks to human society. Adversarial vulnerability refers to the degradation or loss of functionality in deep learning systems under minor perturbations or adversarial conditions. For instance, autonomous driving systems can be significantly affected by abnormal weather such as rain, snow, or fog, causing severe misbehaviors and endangering driver safety (Zhang et al., 2022).

A common *de facto* to evaluate the DNNs is facilitating the humanly labeled tests (Liu et al., 2022; Hu et al., 2023; Li et al., 2024). Nonetheless, crafting adversarial inputs to reveal hidden vulnerabilities in DNNs poses significant challenges for human developers, which demands the necessity of the automated generation of adversarial test cases. In contrast to adversarial attacks, DNN testing exposes model defects and indicates the testing adequacy simultaneously, thereby providing developers with confidence in model deployment (Demir et al., 2020; Guo et al., 2018; Riccio and Tonella, 2020; Pei et al., 2019; Zohdinasab et al., 2023). Thus, DNN testing creates new tests through minor

mutations of seeds from the original data set, aiming to maintain consistent behaviors of model and maximize its test coverage. Therefore, developers are capable of facilitating DNN testing to expose the model to adversarial environments and evaluate its robustness (Wicker et al., 2018; Zhang et al., 2018; Feng et al., 2020; Zohdinasab et al., 2023; Tian et al., 2018; Deng et al., 2022).

Traditional software testing utilizes code coverage to assess the thoroughness of a given test suite, which has been extended to DNN testing. A diverse range of Coverage-Guided Testing (CGT) approaches for DNN have been introduced to generate high-quality test cases (Pei et al., 2019; Ma et al., 2018; Odena et al., 2019; Yuan et al., 2022, 2023; Zhi et al., 2024). DeepXplore (Pei et al., 2019) introduced the first structural-coverage metric particularly designed for DNN testing, known as neuron coverage (NC), which evaluates the ratio of activated neurons to the total. Based on this insight, DeepGauge (Ma et al., 2018) proposed a comprehensive series of neuron-based coverage metrics with different granularity, demonstrating enhanced proficiency in identifying DNNs defects. In recent studies, Yuan et al. (2023)

* Corresponding authors.

E-mail addresses: asye@wtu.edu.cn (A. Ye), 2315363114@mail.wtu.edu.cn (S. Zhang), bxtang@wtu.edu.cn (B. Tang), kejianpeng@whu.edu.cn (J. Ke), zhaoyiru@whu.edu.cn (Y. Zhao), pt@wtu.edu.cn (T. Peng).<https://doi.org/10.1016/j.jnca.2025.104212>

Received 8 August 2024; Received in revised form 18 April 2025; Accepted 5 May 2025

Available online 24 May 2025

1084-8045/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

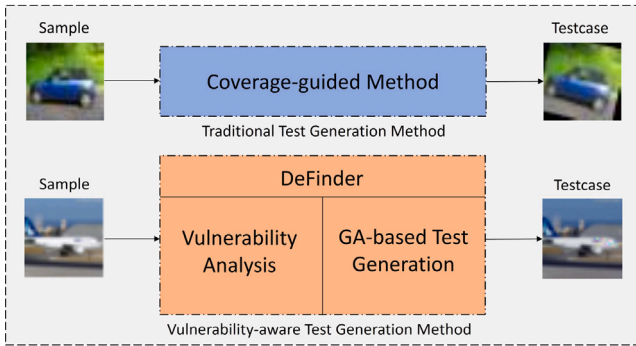


Fig. 1. Comparison between DeFinder and CGT. The first column (top) illustrates the CGT, where the generated test cases are globally mutated to obtain the maximization of test coverage. The second column (bottom) depicts DeFinder performing a vulnerability correlation analysis between seed inputs and target model. The test cases generated by DeFinder exhibit stronger naturalness with minimal perturbations.

proposed a layer-wise neuron coverage considering the layer outputs as an approximation of data distribution. To summarize, test generation driven by test criteria focuses on maximizing coverage and inducing corner-case behaviors. The increment of test coverage signifies more sufficient testing of the model.

Nevertheless, the majority of test coverage metrics are designed to create a more diverse range of test cases. Although this is effective in conventional software testing, it still encounters several challenges in DNN testing: (i) Previous researches (Li et al., 2019; Harel-Canada et al., 2020; Yan et al., 2020; Wang et al., 2021) indicate that neuron coverage is not directly correlated with model vulnerability in DNN testing; (ii) As Fig. 1 illustrates, most of CGT constrain the mutation process via the L_0 , L_1 or L_∞ norm ball to ensure that the generated test meets test oracles (i.e., limiting the number of perturbed pixels or the absolute values of the perturbations to expect that the model decision remains consistent) (Huang et al., 2020). Restricting mutations to a tighter range of pixel space under precise control (coverage) contributes to enhancing confidence in satisfying testing oracles. (iii) The efficacy of DNN testing heavily depends on oracle generation (e.g., metamorphic relations). However, the mechanisms underlying DNN misbehaviors require further elucidation. To this end, it is worth investigating whether interpretable DNNs that illuminate the adversarial vulnerabilities inherent in models. Prior researches (Yuan et al., 2022; Fan et al., 2022) have explored the usage of explainable deep learning frameworks to formulate the oracle problems in DNN testing. Since interpretable models deliver intuitive evidence for model decisions, which aids in connecting tests to model misbehaviors.

To address the aforementioned challenges, we introduce **Defect Finder** (DeFinder), an error-sensitive DNN testing tool via interpreting adversarial vulnerabilities. DeFinder utilizes Explainable AI (XAI) techniques to analyze vulnerability representations within the pixel space of inputs. These inputs are crafted using a collection of adversarial methods. Then identify the fragile regions of inputs to adversarial attack and constrain the mutation to the limited areas. DeFinder employs a vulnerability-aware genetic algorithm to optimize and generate tests that effectively unveil the model defects via maximizing error triggering ratio and test coverage, which provides high confidence for developers. DeFinder provides multiple advantages in DNN testing: Firstly, DeFinder analyzes a series of attack methods that expose model defects, then maps the model misbehaviors to the pixel space through an interpretable framework (i.e., identifying the high-contribution pixels influencing model decisions); Secondly, the test cases generated by DeFinder demonstrate tighter perturbations compared to traditional mutation-based methods, which enhances confidence in meeting test oracles; Thirdly, Compared to merely coverage guided testing, DeFinder induce

more up to 58% erroneous behaviors, which enhances overall testing efficiency. Experiments on nine DNN models using two commonly used datasets have confirmed the effectiveness of DeFinder, demonstrating improvements in test adequacy evaluation and erroneous behaviors detection.

In this work, we propose a novel error-sensitive testing method for DNN called DeFinder to expose the hidden defects in deep learning model. The contributions of this work are summarized as follows:

- We innovatively propose the error-sensitive DNN testing method that enable testing tool to explain the inherent vulnerability representations with highly-visualized test case generation.
- DeFinder significantly enhances the capability of exposing model defects by analyzing the vulnerable area in inputs. Restricting test case mutations to minimal and sensitive areas enhances testing efficiency.
- We have conducted comprehensive experiments on nine DNNs based on two popular visual datasets, and the results indicate that the test cases generated by DeFinder enhance the error triggering ratio up to 58%.

2. Related work

2.1. DNN testing

In the context of DNN testing literature, numerous studies have designed adversarial test generation methods based on novel coverage metrics. Ma et al. (2018) introduced various fine-grained neuron coverage metrics in DeepGauge, manifesting powerful defect detection capabilities. However, some research show that 100% neuron coverage is easily achieved by a test suite with minor scale (Huang et al., 2020). Tian et al. (2018) leveraged greedy search to improve the neuron coverage, thus to test the vulnerabilities in autonomous driving systems. Sun et al. (2019) drew inspiration from traditional software testing for coverage metrics based on causal relationships between neurons in adjacent layers of DNNs. Gopinath et al. (2018) firstly implemented symbolic execution in DNN testing, which transforms DNNs into programmatically executable formats, interpreting activation functions as conditional branches to identify test cases. In summary, these approaches still retain several key insights from traditional software testing.

Coverage-Guided Testing. Odena et al. (2019) introduced TensorFuzz, a methodology that leverages a hill-climbing algorithm to navigate the TensorFlow graph's valid input space, uncovering numerical discrepancies in DNN models. Guo et al. (2018) developed DLFuzz, a neuron coverage-based test case generation technique that outperforms DeepXplore by producing significant 584.62% more test inputs in less time. Xie et al. (2019) presented DeepHunter, a metamorphic mutation-based fuzz testing framework that employs neuron coverage criteria such as KMNC (k -Multisection Neuron Coverage), NBC (Neuron Boundary Coverage), SNAC (Strong Neuron Activation Coverage), TKNC (Top- k Neuron Coverage), TKNP (Top- k Neuron Pattern) to direct the generation of test cases. Kim et al. (2019) proposed metrics to assess the impact of test inputs on training datasets by measuring how surprise they are. Byun et al. (2021) estimated the coverage the subsets of manifold space. Ji et al. (2023) investigated the cause and effect relations between neurons with causality-aware criterion. Liu et al. (2022) adapted the test selection methodology to recurrent neural networks to optimize computation cost. Yuan et al. (2023) introduced the latest state-of-the-art work of neuron-based coverage metric called NLC (Neural Coverage) that considering the outputs of DNN layers as approximate mappings of data distributions. Although CGT methods demonstrate strong capabilities in DNN testing, there is still potential for improving the ability to expose defects.

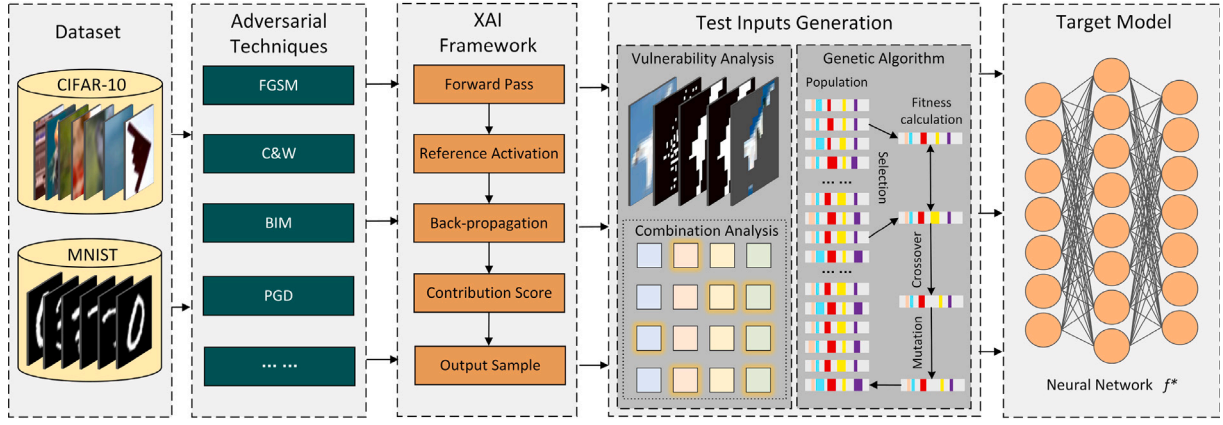


Fig. 2. Overview of DeFinder.

2.2. Adversarial attacks

Adversarial attacks is a critical security threats to DNN-driven systems. Numerous studies demonstrate that the DNNs have high susceptibility to adversarial examples (AEs) (Wang et al., 2024). As the formula (1) shows, the x denote the original input, and y is its corresponding label. The classification model f^* represents the pre-trained DNN model. The perturbation δ is an additive noise that aimed at perpetuating the input x . The goal of AEs is to determine the norm of δ_p is minimized while ensuring the perturbed input $x + \delta$ is misclassified by model f^* .

$$\arg \min_{\|\delta_p\|} f^*(x + \delta) \neq y \quad (1)$$

Goodfellow et al. (2015) attribute AEs to the linear properties of DNN in high-dimensional spaces. Papernot et al. (2016) evaluated the influence of input perturbations on neural network outputs, which represents the forward computation-based adversarial attack. Deep-Fool (Moosavi-Dezfooli et al., 2016) iteratively influences the original sample towards the decision boundary until the misclassification. For malware detection, Grosse et al. (2017) modified the JSMA algorithm to prioritize code feature perturbations using the Jacobian matrix, which incorporates the most critical feature until the detection is effectively bypassed. To make AEs more applicable in real-world scenarios, Huang et al. (2023) proposed a naturalness-aware regularization method that enhances the authenticity of AEs.

3. Methodology

As illustrated in Fig. 2, DeFinder involves several crucial steps. Firstly, we utilize multiple adversarial approaches to provide a variety of AEs. These AEs provide abundant evidence for the vulnerability analysis of model from different perspectives. Then we employ the XAI framework to analyze the collected AEs to accurately locate the vulnerable areas. Thereby, DeFinder is capable of employing the proposed vulnerability-aware genetic algorithms which enhances test coverage and error detection simultaneously in vulnerable areas. Consequently, we input the generated test cases into the target deep neural network and observe the feedback and behaviors.

3.1. Problem formulation

We define the adversarial vulnerability of target model f^* by taking the problem as an indicator function $Error(f^*, x, y)$ whether the output $f^*(x)$ for a given perturbed input $x' = x + \delta$ deviates from the original label y . Here, we constrain the perturbation δ to an L_p norm $\|\delta\|_p < \epsilon$. The f^* is considered as vulnerable under the given input x if there exists δ leading to $f^*(x + \delta) \neq y$, formalized as:

$$Error(x, y, f^*) = \begin{cases} 1 & \text{if } \exists \delta, 0 < \|\delta\|_p < \epsilon \wedge f^*(x + \delta) \neq y \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

To test the adversarial vulnerability of f^* , the objective involves maximizing the count of revealed erroneous behaviors $Error(T, f^*)$ over a test suite $T = \{t_1, t_2, \dots, t_k\}$, in conjunction with the test adequacy metrics $Cov(T, f^*)$ which ensures the target model f^* is sufficiently tested. The testing process is described in Eq. (3).

$$\arg \max_T \left(\sum_{(x,y) \in T} Error(x, y, f^*) + \lambda \cdot Cov(T, f^*) \right) \quad (3)$$

The aforementioned process iteratively generates test cases and refines the test suite to expose and reveal the hidden vulnerabilities in f^* , with λ represented as a weighting parameter to balance the errors count and test adequacy.

3.2. Localizing vulnerable regions

Initially, we analyze AEs generated by four representative attack methods: FGSM (Goodfellow et al., 2015), BIM (Kurakin et al., 2017), C&W (Carlini and Wagner, 2017), and PGD (Madry et al., 2018), which enables a comprehensive analysis of adversarial vulnerability in pixel space. Here, we denote the original input as 'o', while the AEs generated via adversarial perturbations are designated as 'a' (FGSM), 'b' (BIM), 'c' (PGD), and 'd' (C&W). Then, we implement the DeepLIFT (Shrikumar et al., 2017) framework to perform an interpretative analysis of the decision-making process in f^* with input as AEs generated by aforementioned methods. The DeepLIFT assigns the contribution scores s_i to each pixel p_i in input to accentuate its influences on the f^* predictions. This allows us to leverage DeepLIFT to obtain the magnitude of each pixel's influence on the erroneous decision of f^* . To accurately localize the significant pixels, similar with Yuan et al. (2022), the Otsu method (Otsu, 1979) is utilized for automatic thresholding. Succinctly, Otsu algorithm aims to identify a threshold that minimizes the variance within classes while maximizing the variance between classes, thereby achieving a theoretically optimal threshold at reasonable costs. This threshold further aids in filtering out pixel outliers. Then, applying the morphological operations closing and opening operations from OpenCV (Bradski, 2000) to connects minor discontinuities and eliminating the isolated pixels.

We collect a comprehensive range of AEs based on various attack methods for vulnerability analysis. This strategy increases the comprehensiveness of vulnerability analysis and lessen the effects of depending solely on one adversarial hypothesis. Subsequently, the method identifies the intersection of vulnerable regions with the selected combination of AEs, facilitating the precise delineation of vulnerability regions in the sample. The process is described in Formula (4).

$$\mathcal{V} = \bigcap_{k=1}^{|A|} \mathbb{T}(\phi(\mathcal{A}_i(x, f^*))) \quad (4)$$

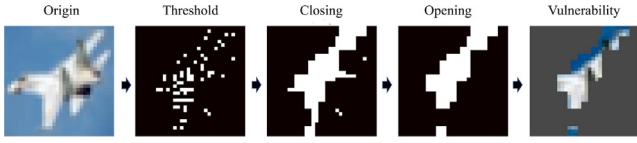


Fig. 3. The illustration of the process of vulnerability localization. As shown above, the vulnerability analysis locates the pixels that highly contribute to model decisions via the implementation of XAI framework, then identifies the regions through morphological operations and generates image masks for seed samples.

where the vulnerability region $\mathcal{V}$ is derived through a composite process involving the generation of AEs from an input sample x against a given neural network f^* , followed by vulnerability analysis leveraging the XAI framework and morphological operations. Specifically, the AEs are generated through leveraging n distinct adversarial attack methods $\mathcal{A} = \{\mathcal{A}_1, \mathcal{A}_2, \dots, \mathcal{A}_k\}$, where each method applied to the input x and model f^* yields a set of AEs which represented as x_{adv}^k . Then we interpreters the generated AEs with the XAI framework ϕ , resulting in a set of explanations (i.e. the most decision-influential pixels detected in these AEs) for each x_{adv}^k . We apply the morphological closing and opening operations to the intersection of $\phi(\mathcal{A}_i(x, f^*))$, which produces transformed sets $\mathbb{T}(\phi(\mathcal{A}_i(x, f^*)))$. The vulnerability region $\mathcal{V}$ is defined as the intersection of all transformed sets, encapsulated mathematically as $\mathcal{V} = \bigcap_{k=1}^{|\mathcal{A}|} \mathbb{T}(\phi(\mathcal{A}_i(x, f^*)))$. As illustrated in Fig. 3, This method emphasizes the potential vulnerable regions of the input sample by analyzing adversarial perturbation, explanation, and transformation.

3.3. Vulnerability-aware tests generation

A critical challenge in testing vulnerabilities of neural networks is to efficiently identify and expose model flaws, especially with limited time and computational resources. Distinct with the adversarial attacks methods, generating adversarial test cases require a thorough and diverse exposure of the erroneous behaviors (i.e., high test coverage and fault detection of model). To address this, DeFinder utilizes the genetic algorithm to enhance the test case generation process by jointly optimizing the DNN testing process, which is proved to be effective in traditional software testing (Esnaashari and Damia, 2021).

We present the Vulnerability-Aware Genetic Algorithm (VAGA) that created to efficiently explore the input space via a natural selection strategy. As described in Algorithm 0, VAGA is capable of identifying input regions that triggering model defects while enhancing the test coverage, thereby facilitating the generation of test cases. Different from original genetic algorithms, VAGA not only refines test coverage but also significantly improves the likelihood of revealing defects in the target model. The principle of VAGA is to mutate the seed inputs within the detected vulnerable regions to obtain optimal adversarial test cases.

Initialization. In DeFinder, we utilize a strategy that obtaining random start points from the original seeds for initializing the population, which aims to create a more wide variety of starting points in the search space, and enhances the likelihood of discovering superior individuals. The perturbations are defined by Eq. (5):

$$x_i = I + \sum_{j=1}^{|Z|} (\alpha_j \cdot \eta) \cdot \mathcal{V}_i \quad (5)$$

s.t. $\forall i \in \{1, \dots, N\}, \alpha_j \in [-1, 1], \eta \in [0, 1]$

where x_i represents each individual in the population, I denotes the original image, $|Z|$ is the total count of perturbable pixels in image I . Each pixel j has a perturbation intensity bounded by parameter α_j . The η represents a uniformly distributed random number in the interval $[0, 1]$, and $\mathcal{V}_i$ specifies the vulnerable region for each individual.

Fitness Function. We quantitatively evaluate the quality of each individual in population by fitness function. The fitness function is

Algorithm 1 Vulnerability-Aware Genetic Algorithm

Require:

- $seed$: The original seed input.
- $\mathcal{V}$: The binary mask of detecting vulnerability.
- f^* : The target DNN.
- pop_size : The size of population.
- $num_generations$: The number of generations to iterate.

Ensure:

- x_{adv} : The optimized adversarial test case.
- 1: $pop = initializePopulation(seed, \mathcal{V}, pop_size)$
 $\triangleright$ Initialize the population with random perturbations of $seed$ within vulnerable areas $\mathcal{V}$.
- 2: **for** $g = 1$ **to** $num_generations$ **do**
 $\triangleright$ Main loop iterating through generations.
- 3: $fitness = get_fitness(f^*, pop)$
 $\triangleright$ Evaluate fitness of each individual in pop using DNN f^* .
- 4: $parents = select_parents(pop, fitness, pop_size/2)$
 $\triangleright$ Select top individuals based on fitness to be parents for crossover.
- 5: $offspring = crossover(parents, pop_size, \mathcal{V})$
 $\triangleright$ Generate new offspring through crossover of selected parents.
- 6: $offspring = mutate(offspring, \mathcal{V})$
 $\triangleright$ Mutate offspring within vulnerable areas $\mathcal{V}$.
- 7: $pop = population_update(parents, offspring)$
 $\triangleright$ Update the population with new offspring.
- 8: **end for**
- 9: $best_idx = argmax(fitness)$
 $\triangleright$ Find index of the individual with highest fitness.
- return** $pop[best_idx]$
 $\triangleright$ Return the best individual found during the optimization process.

defined by combining two critical indicators of the number of induced misbehaviors $Err(x_i, y_i, f^*)$ and the neuron coverage $Cov(x_i, f^*)$.

$$Err(x_i, y_i, f^*) = \max_{y_i \in Y} (\mathbb{P}(f^*(x_i) \neq y_i)) \quad (6)$$

Specifically, $Err(x_i, y_i, f^*)$ involves measuring the prediction error probability of an input x_i , that is, the maximum probability of the model f^* predicting an incorrect category y_i for x_i . $\mathbb{P}(f^*(x_i) \neq y)$ denotes the probability that the f^* predicts any category other than y , calculated as the choice with the maximum prediction probability among all possible predictions in Y .

$$Cov(T, x) = \frac{|\{n \mid \exists x \in T, output(n, x) > \xi\}|}{\sum_{i=0}^k |\{n \mid n \in f^*\}|} \quad (7)$$

Here, the $Cov(x_i, f^*)$ is calculated based on a threshold ξ , where n represents a individual neuron in the model f^* , and n is considered to be covered state when its output exceeds the activation threshold ξ . The accurate process is defined in Eq. (7). Neuron coverage represents the proportion of activated neurons to the total in the DNN under test. The space defined by neuron coverage describes the operational state of the DNN, and the extent of covering this space reflects the thoroughness of the testing. In DeFinder, we select the neuron coverage as indicator mainly based on two factors: the computational cost and synergy with vulnerability analysis. By combining neuron coverage as a joint optimization objective in the fitness function, VAGA is able to generate samples that enhance the sufficiency of the testing. Then we denote the fitness function as follows:

$$f(x_i) = \lambda_e \cdot Err(x_i, y_i, f^*) + \lambda_{cov} \cdot Cov(x_i, f^*) \quad (8)$$

The coefficients λ_e and λ_{cov} weight the contributions of $Err(x_i, y_i, f^*)$ and $Cov(x_i, f^*)$ respectively, optimizing the balance between error detection and coverage.

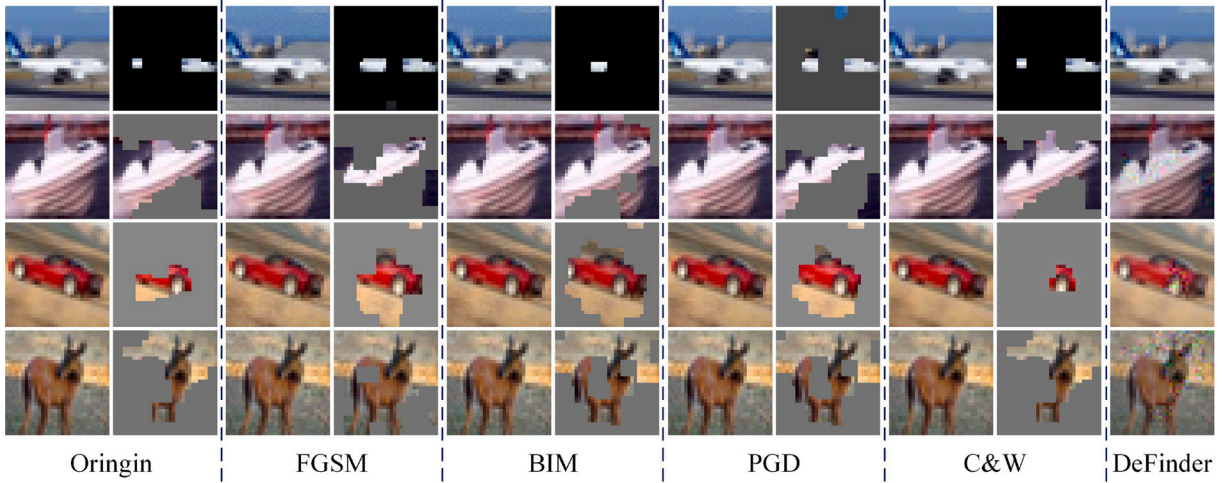


Fig. 4. Display of vulnerable analysis on inputs. The vulnerable regions analyzed within samples from original dataset, FGSM, BIM, PGD, and C&W are shown respectively, The configurations for adversarial attacks are as follows: FGSM with $\epsilon = 0.03$; BIM with $\epsilon = 0.03$, $\alpha = 0.008$, $steps = 10$; PGD with $\epsilon = 0.03$, $\alpha = 0.008$, $steps = 10$; and C&W with $c = 1$, $\kappa = 0$, $steps = 50$, $lr = 0.01$.

Table 1

The table below illustrates the variance in fault detection ratio between different CGT methods and two distinct combinations of DeFinder. The gray-colored cells indicate the outstanding performance compared to other methods.

Models	NC	NBC	SNAC	TKNC	TKNP	KMNC	NLC	GD	STD	DeF_{oab}	DeF_{oabd}
ResNet18	116/200	160/200	124/200	114/200	128/200	177/200	175/200	175/200	173/200	176/200	180/200
ResNet34	112/200	158/200	123/200	109/200	126/200	173/200	169/200	176/200	172/200	178/200	182/200
VGG13_bn	110/200	157/200	123/200	111/200	123/200	177/200	173/200	173/200	171/200	174/200	182/200
VGG16_bn	121/200	163/200	126/200	114/200	127/200	174/200	173/200	176/200	174/200	174/200	180/200
MobileNet_V2	115/200	162/200	122/200	117/200	128/200	176/200	170/200	177/200	169/200	176/200	182/200
DenseNet121	120/200	163/200	127/200	119/200	125/200	174/200	172/200	175/200	170/200	176/200	178/200
DenseNet161	117/200	159/200	127/200	120/200	129/200	176/200	173/200	176/200	172/200	174/200	178/200
GoogLeNet	117/200	159/200	123/200	113/200	131/200	175/200	177/200	178/200	168/200	180/200	182/200

Selection. VAGA employs an elite selection strategy to enhance the parent selection phase, which effectively ensures the propagation of superior genetic inherit across generations. This strategy ensures reliable transmission of high-performing genetic information to the next generations. A vital advantage of VAGA is the ability to preserve the optimal solution for maintaining population, despite the stochastic nature of the selection process, thereby accelerating the convergence of VAGA and increasing the ability of identifying the global optimum. Moreover, elite selection exerts a natural selection pressure on the population, promoting evolutionary progress towards the regions with higher fitness evaluation.

Crossover. VAGA develops a targeted crossover strategy focusing on vulnerable regions to create highly fit offspring. This strategy not only facilitate the generation of adversarial test cases but also increases the sample naturalness. Initially, a mask composed of a set of pixels mask $Mask_{(i,j)}$ is defined to represent vulnerable regions. Each $Mask_{(i,j)}$, located at row i and column j , marks positions where the value is 1 as regions in the individual's genes that are subject to crossover operations. Then we define the indicator function S as follows that returns a specific value based on $Mask_{(i,j)}$ and the parent indicator k :

$$S(Mask_{(i,j)}, k) = \begin{cases} 1, & \text{if } (Mask_{(i,j)} = 1 \cap k = 1) \\ & \cup (Mask_{(i,j)} = 0 \cap k = 2) \\ 0, & \text{otherwise} \end{cases} \quad (9)$$

The generation of offspring δ_c is defined as the following formula (10) demonstrates. Here, i and j designate the horizontal and vertical coordinates of the pixel in the k th parent δ_{pk} . The advantage of VAGA is allowing the offspring to inherit features from the parent in vulnerable region while retaining stable properties in other regions.

$$\delta_c = \sum_{i=1}^m \sum_{j=1}^n \delta_{pk} \cdot S(Mask_{(i,j)}, k) \quad (10)$$

Mutation. VAGA augments population diversity via strategic mutations, enabling a thoroughly exploration of the solution space. It adopts a mutation strategy that selectively introduces random mutations to the vulnerable regions identified in the offspring. For a given set of offspring individuals $\{x_c^1, x_c^2, \dots, x_c^k\}$ and the corresponding vulnerable region $\mathcal{V}_i$, the mutation operation in each offspring individual x_c^i is defined as follows:

$$x_c^i = x_c^i + \sum_{j=1}^{|\mathcal{V}|} (\alpha_j \cdot \eta) \cdot \mathcal{V}_i \quad (11)$$

s.t. $\forall i \in \{1, \dots, N\}, \alpha_j \in [-1, 1], \eta \in [0, 1]$

The approach ensures mutations are limited to the vulnerable region $\mathcal{V}_i$. By concentrating mutations in the specific areas, it not only exposes potential vulnerabilities in the DNNs but also contributes to increasing the robustness. Furthermore, the mutation in VAGA enhances population diversity within the algorithm and avoids mutating non-vulnerable areas, thereby maintaining the natural characteristics of seed samples and reducing semantic distortions.

Population Update. VAGA integrates the newly created offspring into the existing population, organizing them based on fitness evaluations. Subsequently, individuals demonstrating the highest fitness are selected to form a new population for the next evolutionary cycle.

Generally, VAGA explores the vulnerable regions in seed inputs, restricting mutators to operate within the designated areas. VAGA terminates the algorithm based on predefined rules, including reaching the maximum number of iterations or test coverage (e.g., setting the maximum number of iterations to 20 in this case).

4. Experiments

In Section 4, to provide quantitative analysis of DeFinder, we present the experimental results via answering two research questions.

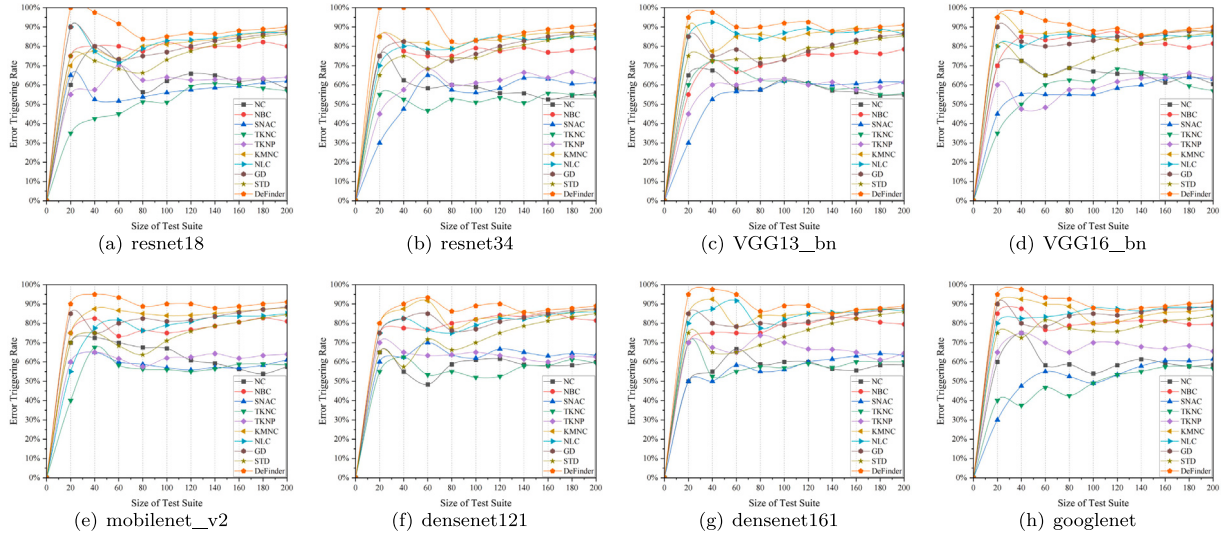


Fig. 5. Error triggering rate across eight DNNs. DeFinder exhibits a markedly superior ability to detect erroneous behaviors compared to baselines, given an identical scale of the test suite. Here, the x-axis represents the size of the test suite measured by the number of test cases (unit: cases).

RQ1 (Vulnerability Interpretation): Can the vulnerability in the inputs be explained? **RQ2 (Revealing Errors in DNNs):** Does vulnerability localization enhance the ability to expose model faults?

4.1. Experiment setup

Datasets. The experiments utilize two popular datasets: CIFAR-10 and MNIST. CIFAR-10 is a computer vision dataset employed for image classification challenges, includes 60,000 color images of 10 different categories, segmented into 50,000 training images and 10,000 test images, each with a resolution of 32×32 pixels. MNIST has a reputation for handwritten digit recognition, which contains 70,000 grayscale images, allocating 60,000 for training and 10,000 for testing, depicting digits from 0 to 9 with a resolution of 28×28 pixels. For this work, 200 images from each dataset are randomly selected as the seed pool.

Model under Test. To thoroughly validate our proposed method, we utilize nine advanced deep learning models trained on two computer vision datasets: ResNet18 and ResNet34 (He et al., 2016), VGG13_bn and VGG16_bn (Simonyan and Zisserman, 2015), MobileNet_V2 (Sandler et al., 2018), DenseNet121 (Huang et al., 2017) and DenseNet161 (Huang et al., 2017), GoogLeNet (Szegedy et al., 2015), and LeNet5 (Lecun et al., 1998). All models were rigorously trained, achieving test accuracies exceeding 94%, with LeNet5 reaching 99%.

Baselines. To demonstrate the efficacy of DeFinder, we select several prevalent coverage metrics including NC, KMNC, NBC, SNAC, TKNC, TKNP, NLC and two black-box testing GD and STD (Aghababayan et al., 2023) as baselines. Then utilize a fuzzing tool with sequentially sampling and the same mutator as DeFinder to create test cases. KMNC defines the major function regions of neurons, then segments the output into k sections. NBC and SNAC emphasize covering corner-case situations of neuron states. TKNC and TKNP focus on evaluating the diversity and frequency of activation patterns. NLC (Yuan et al., 2023) is the latest proposed state-of-the-art neuron coverage to deduce DNN misbehaviors based on approximating layer output distribution.

Configurations. For the experiments, the settings are as follows: the threshold of NC stands at 0.6, which is considerably difficult to be covered, the k in KMNC is set as 1000 to represent the minor-granularity metric, the k in TKNC and TKNP are set as 10 and 50 since the number of parameters in target models is large. The hyper parameters of δ_p are set as α to 0.2 and β to 0.4. The experiments conducted in this work employ the python libraries such as Pytorch, numpy and torchattacks, running on a laptop with an AMD Ryzen 7745HX 3.6 GHz CPU, 16 GB of RAM, and an NVIDIA GeForce RTX 4060 GPU with 8 GB of VRAM.

4.2. RQ1. Vulnerability interpretation

Before analyzing the triggers behind DNNs' erroneous behaviors, the critical research question is, "Do the misbehaviors correlate with the seed inputs?" Although the existence of AEs supports this proposition, another question follows: "Is it possible to interpret the DNNs' corner-case decisions?" To address these questions, the experiments utilize the XAI framework to identify the vulnerable regions by calculating the pixels that contribute to the model's misclassifications. To eliminate bias introduced by selecting representative adversarial attack methods, we employ four popular attacks: FGSM, BIM, PGD, and C&W. We then examine the combinations of these attack methods to present possible interpretations of model vulnerability. As illustrated in Fig. 4, test cases generated by DeFinder demonstrate subtle perturbations compared to AEs. DeFinder limits perturbations to minimal areas, which reduces the distortion of the semantics in seed inputs. The findings from our experimental results, detailed in Table 2, reveal the different phenomena attributed to combinations of attacks. Compared to the baseline 'o', DeFinder contributes to a minor increase in test coverage, with the combinations of 'oabd' and 'oab' producing more distinct outcomes. Additionally, 'oabd' and 'oab' outperform other combinations in inducing erroneous decisions in the target models. This phenomenon does not appear from the combination of 'oa' or 'ob'. A potential explanation could involve the inherent vulnerability association between inputs and the gradient perturbations introduced by FGSM and BIM, since both are gradient-driven adversarial attacks.

Finding 1. Compared with the baselines, DeFinder with NC retains the test coverage within a test suite of 200 cases. An average increase in test coverage of approximately 2% indicates that enhancing capability of fault detection does not compromise test adequacy.

4.3. RQ2. Revealing errors in DNNs

To assess the effectiveness of the test suite in revealing the target model's errors, the efficacy of test suite is quantified by the error-triggering rate for a given scale (i.e., the proportion of test cases that successfully induce errors). Table 1 compares DeFinder with various sophisticated CGT methods and two advanced diversity-guided black-box methods (GD and STD) highlighting the capability of DeFinder to generate effective test cases (Aghababayan et al., 2023). Compared

Table 2

Neuron coverage result for different combinations. Here, the letter o represents the original seed sample with NC optimization, and the letters *a*, *b*, *c*, *d* represent the adversarial attacks FGSM, BIM, PGD, and C&W, respectively. The letter D denotes the proposed DeFinder. We employ various combinations in DeFinder to thoroughly analyze the outcomes of vulnerability analysis. Experimental findings show that the ‘oabd’ and ‘oad’ combinations significantly improve coverage, and none of combination leads to a reduction. The threshold ξ for neuron coverage is set to 0.6.

Model	Ori.	D_{oa}	D_{ob}	D_{oc}	D_{od}	D_{oab}	D_{obc}	D_{ocd}	D_{oad}	D_{oabc}	D_{obcd}	D_{oacd}	D_{oabd}	D_{oabcd}
CIFAR-10														
ResNet18	82.49%	83.72%	83.76%	83.76%	83.80%	84.43%	83.14%	83.80%	84.07%	83.66%	83.76%	83.76%	84.57%	83.74%
ResNet34	80.27%	82.27%	83.83%	83.21%	82.81%	82.18%	83.77%	83.36%	82.73%	82.96%	83.29%	83.02%	83.21%	83.34%
VGG13_bn	60.25%	62.12%	60.85%	62.32%	61.54%	61.87%	62.26%	61.90%	62.16%	62.58%	62.15%	61.59%	61.62%	61.39%
VGG16_bn	61.96%	65.11%	65.14%	65.41%	64.26%	63.89%	63.24%	64.32%	65.92%	63.20%	64.86%	64.34%	65.32%	63.25%
MobileNet_V2	92.12%	93.86%	93.86%	93.82%	93.94%	93.61%	93.11%	93.71%	93.68%	93.74%	93.73%	93.76%	94.54%	92.05%
DenseNet121	76.33%	78.20%	77.38%	78.39%	78.98%	78.91%	77.49%	78.17%	78.93%	78.74%	77.97%	78.30%	79.00%	77.63%
DenseNet161	67.06%	67.55%	67.27%	68.49%	68.31%	67.90%	68.40%	68.22%	68.61%	67.72%	68.13%	67.92%	67.11%	68.16%
GoogLeNet	62.44%	62.48%	62.47%	62.50%	62.59%	62.32%	62.44%	62.44%	62.61%	62.52%	62.34%	62.40%	62.48%	62.44%
MNIST														
LeNet5	88.56%	94.92%	93.64%	94.92%	94.49%	95.34%	94.49%	95.34%	94.07%	95.34%	95.76%	93.64%	94.07%	95.76%

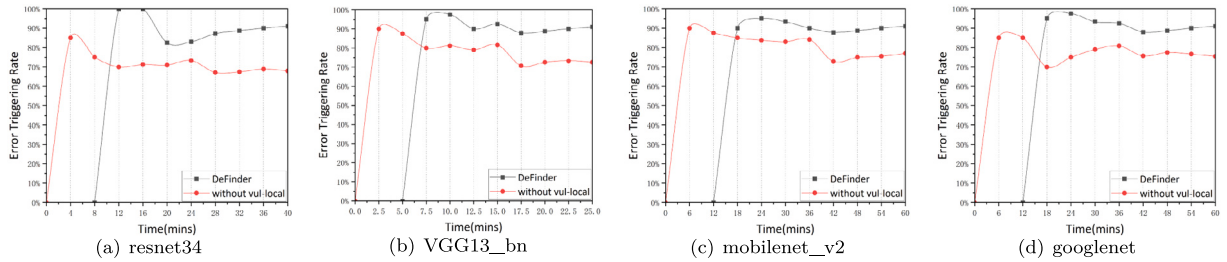


Fig. 6. Compared to the version without the vulnerability localization module (without vul-local), DeFinder generates test cases with a 10%–20% higher error-triggering rate under the same time and computation budget. To ensure a fair comparison, we excluded the time spent of running the vulnerability localization module in DeFinder (i.e., the starting point of DeFinder on the horizontal axis is further to the right).

to obtaining high coverage through global mutations, focusing on mutating the fragile regions increases the error triggering rate without compromising coverage. Experimental results show that DeFinder outperforms in capturing errors compared with baselines, which do not utilize vulnerability analysis to localize fragile areas. The test suite generated by DeFinder achieves a successful error triggering rate as high as 90%, significantly surpassing the approximately 60% ratio of mainstream neuron coverage methods. Fig. 5 illustrates the variation in effectiveness across different test suite sizes, ranging from 20 to 200. DeFinder raise the error triggering rate by up to 58.3% (compared to NC, since we integrates NC in VAGA). Notably, NBC demonstrates a strong defect exposure capacity due to its focus on covering test inputs for unexpected neuron outputs. And GD and STD also demonstrate strong error triggering capability as well as KMNC. NLC, a newly introduced neuron coverage-based CGT, ranks just below DeFinder in error exposure. However, DeFinder still has the potential for joint-optimization with NLC to further enhance its fault detection capacity.

Finding 2. Applying mutations to the restricted pixel space of input is an efficient strategy, DeFinder enhances the probability of exposing erroneous behaviors of model under test by up to 58.3%, compared with baselines.

4.4. Retraining

Using error-triggering test cases for model retraining is widely acknowledged as an effective approach to enhancing the robustness of deep neural networks (Yan et al., 2020). In our experiment, we incorporated error-triggering test cases into the original training set. We subsequently conducted 10 epochs of retraining on each target model. The experimental results are presented in Table 3. After retraining, all target models achieved an average accuracy improvement of about 0.55%, indicating that the model defects exposed by DeFinder contribute to improved model performance.

Table 3

We performed 10 epochs of retraining on the models. The experimental results demonstrate that DeFinder can effectively enhance model robustness.

Models	Before	After	Improved
ResNet18	93.07%	94.13%	1.06%
ResNet34	92.76%	93.33%	0.57%
VGG13_bn	94.21%	94.86%	0.65%
VGG16_bn	94.00%	94.77%	0.77%
MobileNet_V2	93.47%	93.91%	0.44%
DenseNet121	93.66%	94.06%	0.40%
DenseNet161	93.59%	94.08%	0.49%
GoogLeNet	92.47%	92.85%	0.38%
LeNet5	98.79%	98.98%	0.19%

4.5. Ablation experiments

To observe the impact of removing vulnerability localization and genetic algorithm modules of DeFinder, we conducted the ablation experiments on four representative models from each DNN family for demonstrate the efficacy.

For the first scenario, we compared the complete version of DeFinder with a version that excludes the vulnerability localization modules while retaining other components (i.e., the genetic algorithm and mutation operators) which we refer as ‘without vul-local’. Different time budgets were allocated to each model in the experiments due to the impact of model complexity on the computation cost. For instance, when the size of test suite is set as 200, the time budget for VGG13_bn is set to 25 min, ResNet34 to 40 min, and MobileNet_V2 and GoogLeNet to 60 min separately. To ensure fairness in experiments, we extended the runtime of ‘without vul-local’, which compensates for the absence of the vulnerability localization modules. As shown in Fig. 6, with the same time budget, the fault detection rate by using the complete DeFinder is 10%–20% higher than the version of ‘without vul-local’, which demonstrates the effectiveness of the vulnerability analysis module in improving the testing efficiency.

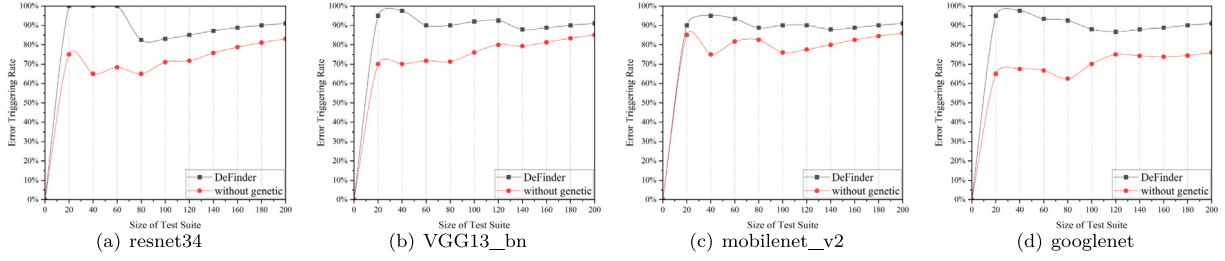


Fig. 7. Compared to the version without the genetic algorithm module (without genetic), DeFinder achieves test cases with approximately 10% higher error-triggering rates.

Table 4

The test coverage across different versions of DeFinder. The threshold ξ for neuron coverage is set to 0.6.

Model	Without genetic	Without vul-local	DeFinder
ResNet34	82.03%	82.47%	83.83%
VGG13_bn	60.88%	61.76%	62.58%
MobileNet_V2	91.74%	92.24%	94.54%
GoogLeNet	62.35%	62.64%	62.61%

Table 5

The time consumption of generating single test case for testing a ResNet18 model with CIFAR-10.

Time cost per case (s)				
NC	NBC	SNAC	TKNC	TKNP
2.38	2.51	2.36	3.67	57.26
KMNC	NLC	GD	STD	DeFinder
9.12	5.22	2.27	2.14	8.3

For the second scenario, we compared the complete DeFinder with a version that excludes the genetic algorithm but retains other components ('without genetic'). The test suite size was set to 200, and the effectiveness of the test suite was quantified using the error triggering rate. As shown in Fig. 7, the error triggering rate of the test suite generated by the complete DeFinder process are 5%–12% higher than the version without genetic module, which indicates the significant efficacy of the genetic algorithm in improving the capability of error exposing.

Moreover, as shown in Table 4, the test coverage of the complete DeFinder are 1%–2% higher than versions of removing vulnerability localization and genetic algorithm modules. The experimental results show that, compared to removing the vulnerability localization module, removing the genetic algorithm module has a higher impact on test coverage. Since the removal of the genetic algorithm module resulted in the lower coverage, showing both modules have a positive influence on test coverage.

4.6. Time consumption

To evaluate the efficiency of DeFinder, we conducted a comparative analysis of time consumption across several testing methods. As illustrated in Table 5, the time overhead of DeFinder is marginally higher than KMNC and TKNP-guided testing tools but remains competitive with other methods. The elevated time cost for KMNC and TKNP can be attributed to the reliance on computationally intensive analyses of neuron importance and functional intervals, which enables them to maintain strong error-revealing capabilities at the expense of efficiency. Conversely, black-box testing methods demonstrate lower time consumption but exhibit limited efficacy in improving coverage due to the absence of model information.

We attribute the performance overhead to two factors: First, the implementation of DeepLIFT for predicting pixel-wise contributions introduces significant runtime costs. Second, the use of genetic algorithms, even guided by NC, inherently involves iterative population updates,

mutation, and evaluation, all of which accumulate computational cost. To address these concerns, we propose three optimization directions for future work: (1) Adopting coverage criteria with lower computational demands, though the weaker synergy with XAI interpretability may incur performance degradation risks; (2) Enhancing the computational efficiency of genetic algorithms through improved offspring selection strategies; (3) Integrating lightweight XAI (e.g., LIME Salih et al., 2025), this demands a new vulnerability detection design for the testing framework.

5. Discussion

DeFinder relies on the interpretability provided by the XAI framework. However, search-based or fuzzing test inherently involves a certain degree of randomness, which makes it difficult to guarantee the generation of effective test cases with a 100% error-triggering rate. Therefore, improving the efficiency of DNN testing tools in triggering errors is of significant value. In our work, DeFinder relies on interpretability frameworks to establish the correlation between pixel space and model decisions. While we cannot fully guarantee that the correlation is as deterministic as the logic of traditional software (deep classifiers are also probabilistic systems), the effectiveness of related interpretability frameworks has been validated in tremendous studies (Shrikumar et al., 2017; Yuan et al., 2022; Angelov and Soares, 2020; Du et al., 2018). As shown in Table 1, our experimental results demonstrate that DeFinder significantly increases the proportion of effective error-triggering test cases in the certain scale of test suite, thereby substantially enhancing the efficiency of the testing tool.

For coverage metrics, the primary rationale for selecting NC stems fundamentally from computational efficiency considerations. NC determines neuron activation based on predefined thresholds, which requires no training set dependency or statistical modeling. This low-cost design is particularly suitable in DeFinder, which already integrates two computationally expensive modules: an XAI framework for vulnerability region localization and a genetic algorithm for test generation.

More advanced structural coverage such as NBC and SNAC require large-scale training data to obtain upper/lower output bounds of neurons, and TKNC/TKNP further depend on top-k sorting across layers that are both significantly increasing computation demands. Similarly, NPC (Xie et al., 2022) involves layer-wise relevance propagation and identifying critical neurons at each layer for decision path abstraction (similar to TKNC but with greater computational overhead), which also observed in works like DeepCNP (Liu et al., 2025). While NLC exhibits lower computational costs than NPC, it still requires computation of layer-level output covariance matrices, maintaining significantly higher complexity than NC. Furthermore, except neuron coverage, alternative approaches like CC (Causality Coverage) construct causal graphs from deep neural networks to enhance interpretability. However, the computational expense associated with causal inference remains high.

We propose that employing XAI frameworks to generate adversarial test cases in DNN testing is highly effective and offers numerous advantages: 1. **Vulnerability**. A principal factor contributing to the vulnerability of neural network decisions to perturbations is attributed to

specific features. Through reverse computation of pixel decision contributions, interpretability frameworks precisely identify features linked to incorrect decisions, which enables the testing of error-sensitive features in DNNs; 2. **Correlation**. Compared to merely globally perturbing inputs, DeFinder provides a new perspective on establishing the correlation between model vulnerabilities and test adequacy; 3. **Visualization**. Using an interpretability framework to analyze model vulnerabilities at the pixel level of inputs provides a clear and visual understanding. The model's vulnerabilities are visualized on the samples to facilitate developer analysis.

6. Conclusions

DeFinder provides a novel approach of DNN testing by focusing on error-sensitive tests through vulnerability interpretation. Unlike traditional CGT methods that heavily rely on designing test criteria via exploring DNN architecture, DeFinder enhances testing efficiency by analyzing the model vulnerabilities through explaining the fragile representations in test inputs. Our experimental results demonstrate that DeFinder significantly increases the proportion of erroneous behaviors that detected by the test suite of limited scale, without compromising test coverage and while preserving sample semantics.

In our future work, we will leverage interpretability frameworks to identify commonalities between model vulnerabilities, and design appropriate test criteria to measure testing adequacy. Furthermore, we will extend interpretability frameworks to the testing of NLP models and implement defect localization analysis for textual data. This will facilitate more comprehensive defect analysis in multimodal models.

CRedit authorship contribution statement

Aoshuang Ye: Writing – review & editing, Writing – original draft, Supervision, Project administration, Conceptualization. **Shilin Zhang**: Writing – review & editing, Writing – original draft, Methodology, Conceptualization. **Benxiao Tang**: Writing – review & editing, Conceptualization. **Jianpeng Ke**: Conceptualization. **Yiru Zhao**: Writing – review & editing. **Tao Peng**: Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the Open Fund of Advanced Cryptography and System Security Key Laboratory of Sichuan Province, China (Grant No. SKLACSS-202313) and Hubei Provincial Natural Science Foundation of China (Grant No.Q20231714 and No.2025AFB323).

Data availability

The data that has been used is confidential.

References

- Aghababaeian, Z., Abdellatif, M., Briand, L.C., Ramesh, S., Bagherzadeh, M., 2023. Black-box testing of deep neural networks through test case diversity. *IEEE Trans. Softw. Eng.* 49 (5), 3182–3204.
- Angelov, P., Soares, E., 2020. Towards explainable deep neural networks (xDNN). *Neural Netw.* 130, 185–194.
- Bradski, G., 2000. The OpenCV library. Dr. Dobb's J. Softw. Tools.
- Byun, T., Rayadurgam, S., Heimdahl, M.P.E., 2021. Black-box testing of deep neural networks. In: Jin, Z., Li, X., Xiang, J., Mariani, L., Liu, T., Yu, X., Ivaki, N. (Eds.), 32nd IEEE International Symposium on Software Reliability Engineering, ISSRE 2021, Wuhan, China, October 25–28, 2021. *IEEE*, pp. 309–320.

- Carlini, N., Wagner, D.A., 2017. Towards evaluating the robustness of neural networks. In: 2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22–26, 2017. *IEEE Computer Society*, pp. 39–57.
- Demir, S., Eniser, H.F., Sen, A., 2020. DeepSmartFuzzer: Reward guided test generation for deep learning. In: Espinoza, H., McDermid, J.A., Huang, X., Castillo-Effen, M., Chen, X.C., Hernández-Orallo, J., hEigeartaigh, S.O., Mallah, R. (Eds.), *Proceedings of the Workshop on Artificial Intelligence Safety 2020 Co-located with the 29th International Joint Conference on Artificial Intelligence and the 17th Pacific Rim International Conference on Artificial Intelligence (IJCAI-PRICA 2020)*, Yokohama, Japan, January, 2021. In: *CEUR Workshop Proceedings*, vol. 2640, CEUR-WS.org.
- Deng, Y., Zheng, X., Zhang, M., Lou, G., Zhang, T., 2022. Scenario-based test reduction and prioritization for multi-module autonomous driving systems. In: Roychoudhury, A., Cadar, C., Kim, M. (Eds.), *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14–18, 2022*. *ACM*, pp. 82–93.
- Du, M., Liu, N., Song, Q., Hu, X., 2018. Towards explanation of DNN-based prediction with guided feature inversion. In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. KDD '18*, Association for Computing Machinery, New York, NY, USA, pp. 1358–1367.
- Esnashari, M., Damia, A.H., 2021. Automation of software test data generation using genetic algorithm and reinforcement learning. *Expert Syst. Appl.* 183, 115446.
- Fan, M., Wei, W., Jin, W., Yang, Z., Liu, T., 2022. Explanation-guided fairness testing through genetic algorithm. In: 44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25–27, 2022. *ACM*, pp. 871–882.
- Feng, Y., Shi, Q., Gao, X., Wan, J., Fang, C., Chen, Z., 2020. DeepGini: prioritizing massive tests to enhance the robustness of deep neural networks. In: Khurshid, S., Pasareanu, C.S. (Eds.), *ISSTA '20: 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, USA, July 18–22, 2020*. *ACM*, pp. 177–188.
- Goodfellow, I.J., Shlens, J., Szegedy, C., 2015. Explaining and harnessing adversarial examples. In: Bengio, Y., LeCun, Y. (Eds.), 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7–9, 2015, *Conference Track Proceedings*.
- Gopinath, D., Wang, K., Zhang, M., Pasareanu, C.S., Khurshid, S., 2018. Symbolic execution for deep neural networks. *CoRR* abs/1807.10439. arXiv:1807.10439.
- Grosse, K., Papernot, N., Manoharan, P., Backes, M., McDaniel, P.D., 2017. Adversarial examples for malware detection. In: Foley, S.N., Gollmann, D., Snekkenes, E. (Eds.), *Computer Security - ESORICS 2017 - 22nd European Symposium on Research in Computer Security*, Oslo, Norway, September 11–15, 2017, *Proceedings, Part II*. In: *Lecture Notes in Computer Science*, vol. 10493, Springer, pp. 62–79.
- Guo, J., Jiang, Y., Zhao, Y., Chen, Q., Sun, J., 2018. Dlfuzz: differential fuzzing testing of deep learning systems. In: Leavens, G.T., Garcia, A., Pasareanu, C.S. (Eds.), *Proceedings of the 2018 ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2018, Lake Buena Vista, FL, USA, November 04–09, 2018*. *ACM*, pp. 739–743.
- Harel-Canada, F., Wang, L., Gulzar, M.A., Gu, Q., Kim, M., 2020. Is neuron coverage a meaningful measure for testing deep neural networks? In: Devanbu, P., Cohen, M.B., Zimmermann, T. (Eds.), *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8–13, 2020*. *ACM*, pp. 851–862.
- He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27–30, 2016. *IEEE Computer Society*, pp. 770–778.
- Hu, Q., Guo, Y., Xie, X., Cordy, M., Papadakis, M., Ma, L., Traon, Y.L., 2023. Aries: Efficient testing of deep neural networks via labeling-free accuracy estimation. In: 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14–20, 2023. *IEEE*, pp. 1776–1787.
- Huang, X., Kroening, D., Ruan, W., Sharp, J., Sun, Y., Thamo, E., Wu, M., Yi, X., 2020. A survey of safety and trustworthiness of deep neural networks: Verification, testing, adversarial attack and defence, and interpretability. *Comput. Sci. Rev.* 37, 100270.
- Huang, G., Liu, Z., van der Maaten, L., Weinberger, K.Q., 2017. Densely connected convolutional networks. In: 2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21–26, 2017. *IEEE Computer Society*, pp. 2261–2269.
- Huang, Y., Sun, L., Guo, Q., Juefei-Xu, F., Zhu, J., Feng, J., Liu, Y., Pu, G., 2023. ALA: naturalness-aware adversarial lightness attack. In: El-Saddik, A., Mei, T., Cucchiara, R., Bertini, M., Vallejo, D.P.T., Atrey, P.K., Hossain, M.M. (Eds.), *Proceedings of the 31st ACM International Conference on Multimedia, MM 2023, Ottawa, on, Canada, 29 October 2023– 3 November 2023*. *ACM*, pp. 2418–2426.
- Ji, Z., Ma, P., Yuan, Y., Wang, S., 2023. CC: causality-aware coverage criterion for deep neural networks. In: 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14–20, 2023. *IEEE*, pp. 1788–1800.
- Kim, J., Feldt, R., Yoo, S., 2019. Guiding deep learning system testing using surprise adequacy. In: Atlee, J.M., Bultan, T., Whittle, J. (Eds.), *Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, Montreal, QC, Canada, May 25–31, 2019*. *IEEE / ACM*, pp. 1039–1049.

- Kurakin, A., Goodfellow, I.J., Bengio, S., 2017. Adversarial examples in the physical world. In: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Workshop Track Proceedings. OpenReview.net.
- LeCun, Y., Bottou, L., Bengio, Y., Haffner, P., 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86 (11), 2278–2324.
- Li, Z., Ma, X., Xu, C., Cao, C., 2019. Structural coverage criteria for neural networks could be misleading. In: Sarma, A., Murta, L. (Eds.), Proceedings of the 41st International Conference on Software Engineering: New Ideas and Emerging Results, ICSE (NIER) 2019, Montreal, QC, Canada, May 29-31, 2019. IEEE / ACM, pp. 89–92.
- Li, Y., Zheng, Y., Teo, Y.S., Lin, S., 2024. Is AI testing beneficial for the manufacturer and social welfare? optimal test strategy of a smart product. *Expert Syst. Appl.* 241, 122637.
- Liu, Z., Feng, Y., Yin, Y., Chen, Z., 2022. DeepState: Selecting test suites to enhance the robustness of recurrent neural networks. In: 44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022. ACM, pp. 598–609.
- Liu, W., Luo, S., Pan, L., Zhang, Z., 2025. DeepCNP: An efficient white-box testing of deep neural networks by aligning critical neuron paths. *Inf. Softw. Technol.* 179, 107640.
- Ma, L., Juefei-Xu, F., Zhang, F., Sun, J., Xue, M., Li, B., Chen, C., Su, T., Li, L., Liu, Y., Zhao, J., Wang, Y., 2018. DeepGauge: multi-granularity testing criteria for deep learning systems. In: Huchard, M., Kästner, C., Fraser, G. (Eds.), Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ASE 2018, Montpellier, France, September 3-7, 2018. ACM, pp. 120–131.
- Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A., 2018. Towards deep learning models resistant to adversarial attacks. In: 6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings. OpenReview.net.
- Moosavi-Dezfooli, S., Fawzi, A., Frossard, P., 2016. DeepFool: A simple and accurate method to fool deep neural networks. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016. IEEE Computer Society, pp. 2574–2582.
- Odena, A., Olsson, C., Andersen, D.G., Goodfellow, I.J., 2019. TensorFuzz: Debugging neural networks with coverage-guided fuzzing. In: Chaudhuri, K., Salakhutdinov, R. (Eds.), Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA. In: Proceedings of Machine Learning Research, vol. 97, PMLR, pp. 4901–4911.
- Otsu, N., 1979. A threshold selection method from gray-level histograms. *IEEE Trans. Syst. Man Cybern.* 9 (1), 62–66.
- Papernot, N., McDaniel, P.D., Jha, S., Fredrikson, M., Celik, Z.B., Swami, A., 2016. The limitations of deep learning in adversarial settings. In: IEEE European Symposium on Security and Privacy, EuroS&P 2016, Saarbrücken, Germany, March 21-24, 2016. IEEE, pp. 372–387.
- Pei, K., Cao, Y., Yang, J., Jana, S., 2019. DeepXplore: automated whitebox testing of deep learning systems. *Commun. ACM* 62 (11), 137–145.
- Riccio, V., Tonella, P., 2020. Model-based exploration of the frontier of behaviours for deep learning system testing. In: Devanbu, P., Cohen, M.B., Zimmermann, T. (Eds.), ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020. ACM, pp. 876–888.
- Salih, A.M.A., Raisi-Estabragh, Z., Galazzo, I.B., Radeva, P., Petersen, S.E., Lekadir, K., Menegaz, G., 2025. A perspective on explainable artificial intelligence methods: SHAP and LIME. *Adv. Intell. Syst.* 7 (1).
- Sandler, M., Howard, A.G., Zhu, M., Zhmoginov, A., Chen, L., 2018. MobileNetV2: Inverted residuals and linear bottlenecks. In: 2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018. Computer Vision Foundation / IEEE Computer Society, pp. 4510–4520.
- Shrikumar, A., Greenside, P., Kundaje, A., 2017. Learning important features through propagating activation differences. In: Precup, D., Teh, Y.W. (Eds.), Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017. In: Proceedings of Machine Learning Research, vol. 70, PMLR, pp. 3145–3153.
- Simonyan, K., Zisserman, A., 2015. Very deep convolutional networks for large-scale image recognition. In: Bengio, Y., LeCun, Y. (Eds.), 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings.
- Sun, Y., Huang, X., Kroening, D., Sharp, J., Hill, M., Ashmore, R., 2019. Structural test coverage criteria for deep neural networks. *ACM Trans. Embed. Comput. Syst.* 18 (5s), 94:1–94:23.
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S.E., Anguelov, D., Erhan, D., Vanhoucke, V., Rabinovich, A., 2015. Going deeper with convolutions. In: IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015. IEEE Computer Society, pp. 1–9.
- Tian, Y., Pei, K., Jana, S., Ray, B., 2018. DeepTest: automated testing of deep-neural-network-driven autonomous cars. In: Chaudron, M., Crnkovic, I., Chechik, M., Harman, M. (Eds.), Proceedings of the 40th International Conference on Software Engineering, ICSE 2018, Gothenburg, Sweden, May 27 - June 03, 2018. ACM, pp. 303–314.
- Wang, Z., Guo, J., Chen, Y., She, F., 2021. The effect of combinatorial coverage for neurons on fault detection in deep neural networks. In: 21st IEEE International Conference on Software Quality, Reliability and Security, QRS 2021 - Companion, Hainan, China, December 6-10, 2021. IEEE, pp. 77–82.
- Wang, H., Wu, C., Zheng, K., 2024. Defense against adversarial attacks based on color space transformation. *Neural Netw.* 173, 106176.
- Wicker, M., Huang, X., Kwiatkowska, M., 2018. Feature-guided black-box safety testing of deep neural networks. In: Beyer, D., Huisman, M. (Eds.), Tools and Algorithms for the Construction and Analysis of Systems - 24th International Conference, TACAS 2018, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2018, Thessaloniki, Greece, April 14-20, 2018, Proceedings, Part I. In: Lecture Notes in Computer Science, vol. 10805, Springer, pp. 408–426.
- Xie, X., Li, T., Wang, J., Ma, L., Guo, Q., Juefei-Xu, F., Liu, Y., 2022. NPC: neuron path coverage via characterizing decision logic of deep neural networks. *ACM Trans. Softw. Eng. Methodol.* 31 (3), 47:1–47:27.
- Xie, X., Ma, L., Juefei-Xu, F., Xue, M., Chen, H., Liu, Y., Zhao, J., Li, B., Yin, J., See, S., 2019. DeepHunter: a coverage-guided fuzz testing framework for deep neural networks. In: Zhang, D., Möller, A. (Eds.), Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019, Beijing, China, July 15-19, 2019. ACM, pp. 146–157.
- Yan, S., Tao, G., Liu, X., Zhai, J., Ma, S., Xu, L., Zhang, X., 2020. Correlations between deep neural network model coverage criteria and model quality. In: Devanbu, P., Cohen, M.B., Zimmermann, T. (Eds.), ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020. ACM, pp. 775–787.
- Yuan, Y., Pang, Q., Wang, S., 2022. Unveiling hidden DNN defects with decision-based metamorphic testing. In: 37th IEEE/ACM International Conference on Automated Software Engineering, ASE 2022, Rochester, MI, USA, October 10-14, 2022. ACM, pp. 113:1–113:13.
- Yuan, Y., Pang, Q., Wang, S., 2023. Revisiting neuron coverage for DNN testing: A layer-wise and distribution-aware criterion. In: 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023. IEEE, pp. 1200–1212.
- Zhang, J.M., Harman, M., Ma, L., Liu, Y., 2022. Machine learning testing: Survey, landscapes and horizons. *IEEE Trans. Softw. Eng.* 48 (2), 1–36.
- Zhang, M., Zhang, Y., Zhang, L., Liu, C., Khurshid, S., 2018. DeepRoad: GAN-based metamorphic testing and input validation framework for autonomous driving systems. In: Huchard, M., Kästner, C., Fraser, G. (Eds.), Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ASE 2018, Montpellier, France, September 3-7, 2018. ACM, pp. 132–142.
- Zhi, Y., Xie, X., Shen, C., Sun, J., Zhang, X., Guan, X., 2024. Seed selection for testing deep neural networks. *ACM Trans. Softw. Eng. Methodol.* 33 (1), 23:1–23:33.
- Zohdinasab, T., Riccio, V., Tonella, P., 2023. DeepAtash: Focused test generation for deep learning systems. In: Just, R., Fraser, G. (Eds.), Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023. ACM, pp. 954–966.